

Spenda v2 - External Security Assessment

service-api.spenda.africa · genesis-ark.spenda.africa · sbi.spenda.africa (BI)

Date: 2026-09-09 (UTC) · **Mode:** External, unauthenticated, non-destructive

Classification: Confidential - for the Spenda security team

Severity ordering: findings are listed strictly in descending severity. Metabase findings lead the report as requested.

Spenda v2, External Security Assessment

Targets: service-api.spenda.africa and genesis-ark.spenda.africa (+ platform BI)

Assessment window: 2026-09-09 (UTC) · **Mode:** External, unauthenticated, non-destructive

Methodology: Passive infrastructure enumeration, unauthenticated API probing, JavaScript bundle analysis, server-action replay, fuzzing and differential testing, CVE ground-truth verification. Credential-based testing used controlled audit accounts or non-existent identifiers only. No funds moved, no real customer data read, no destructive changes.

Findings are ordered **by severity, highest first**. The Metabase findings are presented first at the customer's request because they carry Critical-rated, actively-exploited CVE exposure.

1. CRITICAL, Metabase BI server (sbi.spenda.africa): affected by Critical 2026 advisory with a live-confirmed bypass

The platform's Business-Intelligence server runs **Metabase v0.53.7.1** (OSS, build 2025-03-18), a build now **six major versions behind current**. During 2026 Metabase published a series of Critical advisories; this build sits inside the affected range of one of them and was verified live.

1.1 GHSA-r495-55cx-fjh7, "Multiple vulnerabilities across query validation, permissions, and network exposure", CONFIRMED AFFECTED

- **Published:** 2026-08-11 · **Severity:** Critical
- **Affected:** `>= x.58.0 < x.58.28`, `>= x.59.0 < x.59.25`, `>= x.60.0 < x.60.21`, `>= x.61.0 < x.61.15`, `>= x.62.0 < x.62.13`, `>= x.63.0 < x.63.10`, plus a standalone sweep line `< x.58.28` that includes **v0.53.7.1**.

What this advisory contains (all unpatched on the target):

#	Advisory item	What it means
1	Query-input SQLi hardening	Crafted values in filters, cast options, stored metric/segment definitions, custom column types, and date/regex/percentile/timezone arguments were interpolated as raw SQL into warehouse queries instead of being treated as data. Every one of these code paths exists unpatched on v0.53.7.1.
2	HoneySQL CVE-2026-61620	The SQL-generation library Metabase depends on had its own published vulnerability; the target ships the vulnerable generation path.
3	Nested-card permission bypass	Permission checks verified only the first levels of question nesting, deeply nested references slipped past and surfaced data the viewer should not reach.
4	Login/password-reset rate-limit bypass via header variation	A client-controlled request header resets the per-client throttles, brute-force protection could be switched off at will. Confirmed live on the target (below).
5	Sandboxing/impersonation integrity	Crafted requests could skip row/column restrictions entirely; sandbox-defining questions could be edited by the very users they constrain, allowing self-elevation .
6	Loose API request validation	Endpoints accepted unexpected fields and acted on them (mass-assignment class).
7	Network exposure	A legacy "HTTP action" path let the server issue requests to internal addresses (SSRF); database SSH tunnels were not bound to loopback; an unauthenticated request could influence the instance's configured site URL .
8	Row/download limit override	Clients could raise or remove the server-side result caps.

1.1a Live confirmation performed on the target (ground truth, not banner-matching)

The advisory's item 4 was verified **working right now** on `sbi.spenda.africa`:

```
POST /api/session/reset_password (token probe, 9 rapid attempts)
→ throttle engages: "Too many attempts! You must wait 14 seconds..."

Same request with rotated X-Forwarded-For / X-Real-IP / Client-IP:
→ throttle cleared instantly, 4/4 fresh requests accepted
(control without rotation: stays throttled)
```

Impact. The only brake on token/code guessing against this BI host is fully evadable by header rotation, the guessing surface is **unbounded**. Items 1/2/5 (the SQLi and self-elevation paths) require an authenticated session; combined with unbounded spraying, any low-privilege foothold converts to full application-database read (session records, bcrypt password hashes, and, where `MB_ENCRYPTION_SECRET_KEY` is unset, the default, **cleartext credentials for every connected data warehouse**, i.e., the production analytics store behind this fintech).

Remediation. Upgrade to **≥ 0.58.28 immediately** (ideally current 0.63.10+); until then, enforce strict header normalization at the edge (WAF: single canonical client IP), disable public sharing, and audit query/activity history for past abuse of these paths.

1.2 Critical CVEs the target escapes only by version arithmetic, explained

These are the Critical, actively-exploited Metabase vulnerabilities of 2026. The target is **not** in their affected ranges, but only because the vulnerable code was introduced in versions *newer* than the one deployed. They are documented here because the exposure question recurs on every upgrade:

- **CVE-2026-72898 (CVSS 10.0, CISA KEV, actively exploited since 2026-08-11).** Unauthenticated SQL injection via `POST /api/session/reset_password` leading directly to Metabase **admin takeover**, then theft of stored warehouse credentials. Affects x.58.0 x.63.3. Ground-truth tested on this instance: the

endpoint exists but the token parameter is strictly schema-validated and the lookup is parameterized, all injection payloads return a uniform "Invalid reset token" with zero timing differential. **Verdict: not exploitable at v0.53.7.1**, but any upgrade landing on an x.58-x.63 line *without* the patch deploys this bug live onto an internet-facing host.

- **CVE-2026-59827 / GHSA-w95f-x9v9-wv36 (CVSS 9.9)**. Unsafe deserialization of H2 query results → RCE for any user able to run native queries (the default sample database is H2). Affects $\geq 1.58.0$. Not in range here.
- **GHSA-cwxq-fmxq-jv8h (Critical)**. Arbitrary file read/write via H2 `FILE_READ` / `FILE_WRITE` / `CSVREAD` built-ins. Affects $\geq 1.55.0$. Not in range here.
- **CVE-2026-33725 (Critical, Enterprise)**. RCE via serialization import; Enterprise-only, not applicable to this OSS build.

1.3 In-range Metabase items (lower severity)

- **CVE-2025-30371 (Low, in range < 0.53.8)**: GeoJSON local-link protection bypass (SSRF). Verified present by version; exploitation requires an **admin** session (unauthenticated attempts receive 403 on all internal/IMDS targets). It is the primary post-admin escalation step (instance-local services → IMDS at 169.254.169.254 → cloud credentials).
- **Setup-token disclosure (LOW, hygiene)**: `GET /api/session/properties` publishes the instance setup token (`d58b4c21-...`). Its consumer endpoints were tested: `POST /api/setup` validates the token and then correctly refuses because setup is complete, **no privilege escalation today**, but the token is a loaded gun if the instance is ever reset/redeployed without rotation.
- **oembed iframe reflection (LOW)**: `/api/public/oembed?url=<any>` reflects arbitrary URLs into an iframe on the trusted `sbi.spenda.africa` origin (client-side only; phishing-grade).

2. CRITICAL, Cross-environment JWT signing-key sharing extends to production (service-api.spenda.africa)

Finding. All environments (dev, staging, production) share the same RS256 signing keys. Proof, a JWT signed by the **development** environment (issuer `spenda.user-svc`, expired ~23 hours before the test) is **cryptographically validated by production services**:

Replay target	Response	Meaning
prod user-svc /me	401 {"code":"session.expired"}	Signature verified; session looked up; only TTL failed
prod transaction-svc / transactions	401 {"message":"Signature has expired"}	RS256 signature verified against prod verification keys
staging wallet/balances	401 "Signature has expired"	Same trust across the staging pair
Control: garbage signature	401 "Invalid token"	Distinct signature-failure path, proving the check runs

Impact. The dev and staging estates were previously demonstrated to contain unauthenticated execution paths (fixed verification codes, unauth platform funding, webhook forgeries). Shared keys convert any such compromise, or any leaked lower-environment secret, into a **production session forgery primitive** within token TTL: customer ATO and, for admin-audience tokens, admin access.

Remediation. 1. Separate RS256 keypair per environment; no shared JWKS or secrets. 2. Pin environment-specific `iss/aud` in every service; reject foreign issuers. 3. Per-environment `kid` allow-lists at Kong. 4. Rotate the currently shared keypair immediately, treat it as compromised. 5. Regression test: a dev-signed token must fail on prod with a *signature* error, never an "expired" error.

3. HIGH, Staging backend reachable through the production gateway (Host-header bleed)

Finding. The production gateway IP routes by Host header. Sending `Host: stageapi.spenda.africa` to the **production** IP returns the **staging backend**, confirmed by dataset differential: staging serves 11 FAQ items ("Spenda is a fintech platform..."), production serves 15 ("Spenda is a crypto spending app..."); responses match the real staging host byte-for-byte. `user-svc` staging health also answers through the same path.

Impact. Staging is exposed on the production edge **without its own edge protections**. Every historical staging flaw (unauth KYC reset, MFA-stepup DB errors, no admin lockout) becomes internet-reachable through the prod gateway the moment any staging regression ships.

Remediation. Exact-Host allow-lists at Kong (421 on unknown hosts); physically separate staging and production edges; WAF rule for cross-host references.

4. HIGH, Internal topology disclosed unauthenticated (pod IPs, ports, hostnames, dependency health)

Finding. Two channels:

1. **Slash-redirect leak:** any slash-terminated route 307-redirects with the **internal pod address** - `https://10.0.19.2:9200/...` (`user-svc`), `https://10.0.12.44:9200/...` (`user-svc auth`), `https://10.0.19.15:9100/...` (`ops-svc`), `https://10.0.14.153:8001/...` (`card-svc`).
2. **Health endpoints:** unauthenticated `health/ready` on all five services returns pod hostnames plus dependency states: `database: up`, `redis: up`, and Kafka detail, **brokers: 12; topics: 8-10**, including live outage state (`rates-svc` and `transaction-svc` report `kafka: DOWN`, a visible event-bus failure).

Impact. Complete production map (pods, replicas, ports, event-bus shape) for attack planning; the leaked IPs themselves are externally unreachable (verified, network policy correct), so this feeds the other chains rather than standing alone; the Kafka-outage visibility is operational intelligence.

Remediation. Normalize trailing slashes at Kong (no upstream Location passthrough); authenticate `health/ready` (public endpoint should be a static `healthz`); strip host/dependency detail from external responses.

5. MEDIUM findings (service-api + genesis-ark)

5.1 Reproducible 500 crash on form-encoded authentication

`POST .../admin/auth/login` and `POST .../api/v1/auth/login` with `Content-Type: application/x-www-form-urlencoded` return 500 `app.internal_error` reproducibly (both services), crash occurs **before validation** (missing content-type guard). Log-noise/error-budget tool for attackers; signals a fragile parser. **Fix:** 415 at the schema layer + parser tests.

5.2 Debug endpoints exposed in production

`/api/users/sentry-debug` and `/api/transactions/sentry-debug` return internal error envelopes, unauthenticated. **Fix:** remove from prod builds / environment-gate.

5.3 Admin portal acts as a WAF-bypass proxy

Injection payloads blocked by the WAF when sent directly to `service-api` reach the backend unimpeded when relayed through `genesis-ark`'s server actions (verified both ways). The server-side proxy path skips edge inspection. **Fix:** inspect relayed bodies at the WAF or at the API layer independent of the edge.

5.4 Client-controlled environment switching in the admin portal

The portal selects staging vs production via `localStorage("staging_environment")` → `x-environment` header on `/api/authentication/refresh`. Combined with Finding 2, a staging session is one header away from a production refresh. **Fix:** pin environment server-side per deployment; ignore client hints.

6. LOW findings

- **Setup-token disclosure (Metabase):** published publicly; currently dead-ended by the setup-completed gate (tested), rotate anyway.
- **oembed iframe reflection (Metabase):** arbitrary URL in iframe on the trusted origin (phishing-grade, client-side).
- **Default-port redirect disclosure (ALB):** `http://<ip>/` → 301 to `https://<ip>:443/`.
- **Port 21 artifact:** TCP-accept with no banner/protocol on all estate IPs (AWS-internal artifact; anonymous FTP dead; not exploitable).

7. Verified Resilience (tested and held)

Attack class	Result
SQL injection (login, 2FA, FAQ, ledger params; boolean/time/union/tautology, direct and via portal)	No injection; strict schema + parameterized lookups; WAF blocks direct payloads
JWT attacks (alg:none, kid traversal, HS256 confusion, type confusion)	Uniform "Invalid token"
Kong route confusion (<code>../</code> , <code>%2e%2e</code> , <code>..;</code> , <code>///</code> , case, matrix)	Blocked; no cross-service traversal
Trusted-header auth bypass (12 vectors)	All 401
Verb tampering on POST-only admin routes	Correct 405s
Unauth IDOR (user/transaction/rails detail, by-spenda-id)	All 401
Purpose/audience confusion (email_verify token → admin endpoints)	"Invalid token", aud/purpose enforced
Admin login brute force (portal + API)	1 fail → 10s cooldown → hourly per-email ban + global IP budget; 10-email spray fully throttled
2FA (verify2FAAction)	No oracle; uniform responses
CORS	Strict allowlist; no attacker-origin reflection with credentials
FAQ public endpoint	Strict bounds (0 and >100 → 422); no draft leak; correct pagination meta
Internal ops routes (funding/transfer/ledger/otc)	Not deployed at the prod gateway, the dev-era critical flaw did not propagate
Webhook endpoints (fireblocks/rain)	Absent at prod gateway
Email verification	Fixed-code flaw removed; 5-attempt countdown + device-enroll email code
Edge hardening	

Attack class	Result
	WAF 403s on /.env*, /.aws/*, /.ssh/* with all bypasses blocked (traversal/encoding/case/null-byte); h2c ignored; TRACE/CONNECT stopped

8. Unauthorized Access Achieved (no credentials used)

- 1. Production API surface map (~200 routes)** across all five services with per-route method/auth status, from public JS bundles + schema replay + crawling, including admin-only routes (users/list, finance/accounting/funding, account-moderation/*, control-center/kyc/*, settings/*).
- 2. Two Server Action IDs** (loginAction, verify2FAAction), programmatic portal authentication attempts without a browser.
- 3. Internal pod addresses and ports** via trailing-slash redirects (four pods disclosed).
- 4. Pod hostnames, replica topology, dependency health** for all five services, including Kafka broker/topic counts and live outage states.
- 5. Staging backend responses through the production gateway** (Host bleed) and the staging-vs-production dataset differential.
- 6. Environment fingerprinting**, frameworks, error envelopes, correlation-ID echo, rate-limit bucket sizes, certificate SAN scope.
- 7. Controlled account creation on production** (register → device-enroll step), signup pipeline automation-usable; verify step properly attempt-gated.
- 8. Metabase:** full API surface, setup token, live throttle-bypass PoC, version/build fingerprint.

Blocked (did not achieve): production session forgery (only the 15-minute TTL stands between the shared-key flaw and full forgery, per Finding 2); admin authentication; ledger/funding access (routes absent); real customer data.

9. Open Ports & Edge Exposure

Sweep across all six estate IPs (top 30 ports): **only 21, 80, 443 open.**

Port	State	Assessment
21	TCP-accept, no banner, no protocol (nmap ftp?)	AWS artifact; anonymous dead; not exploitable
80	AWS ELB 301 → https://<ip>:443/	Default-port disclosure (cosmetic)
443	Kong/ALB; Amazon RSA 2048 cert, CN/SAN = service-api.spenda.africa only	No wildcard; no internal names leaked

Edge: WAF dotfile/sensitive-path rules hold against all bypass techniques; no hidden vhosts beyond the stageapi bleed (Finding 3); no metrics/debug exposure; DB/middleware ports (5432/3306/6379/9200/27017/9092) all closed.

10. Prioritized Remediation Plan

Immediate 1. **Metabase ≥ 0.58.28** (ideally 0.63.10+); canonicalize client-IP headers at its edge; disable public sharing meanwhile; audit query history for abuse. 2. **Separate per-environment JWT keys**; rotate the shared keypair (Finding 2). 3. **Default-deny unknown Hosts at Kong**; isolate staging from the prod edge (Finding 3). 4. Authenticate/strip health/ready; normalize trailing slashes (Finding 4). 5. Fix the form-encoding 500 (415 guard); remove sentry-debug (Findings 5.1-5.2).

Near-term 6. WAF inspection for server-action-relayed traffic (5.3); server-pinned environment (5.4). 7. Rotate the Metabase setup token; audit warehouse credentials if `MB_ENCRYPTION_SECRET_KEY` is unset. 8. Regression tests for every row of §7 so the verified controls stay true.

Strategic 9. Zero-trust between environments: separate edges/VPCs/secret stores; no shared JWKS or credentials anywhere. 10. Dependency-hygiene checklist (the root causes here, shared keys, Host routing, verbose health, stale BI, are systemic).

11. Assessment Timeline (UTC, 2026-09-09)

Time	Activity
12:40	service-api identified: Kong 3.9.3, five route prefixes
12:44-12:47	Pod names via health; 172-path schema replay (full status matrix)
13:14-13:35	genesis-ark recon: server-action IDs, env-switching logic, katana crawl
14:14-14:47	Browser-accurate server-action replay; WAF asymmetry; lockout behavior
20:40-21:00	Endpoint battery (34 probes); 2FA oracle testing (negative)
21:17-21:41	Dependency health map; Kafka outage observed
21:41-22:46	Port/vhost sweeps; WAF-bypass battery; nmap confirmation
22:00-23:30	Metabase CVE triage; GHSA-r495 rate-limit bypass confirmed live

External, non-destructive assessment. All PoCs used controlled accounts or non-existent identifiers. Severity follows CVSS-informed judgment on real-world impact.